

Technik Programista

Testowanie i dokumentowanie aplikacji

Git i GitHub

Kontrola wersji – wstęp

Repozytorium, commity, GitHub, README, .gitignore, gh CLI

Klasa 4 | rok szkolny 2026/2027 | System: Windows

Spis treści

Spis treści

Spis treści.....	2
1. Po co to w ogóle jest?	3
2. Model w głowie: trzy miejsca.....	3
3. Instalacja i konfiguracja	4
3.1 Instalacja (raz, na tym komputerze).....	4
3.2 Konfiguracja (raz)	4
4. Konto na GitHubie	4
5. Pierwszy projekt – krok po kroku.....	5
5.1 Utwórz repozytorium na GitHubie	5
5.2 W folderze projektu na komputerze	5
5.3 Sprawdź.....	5
6. Codzienna pętla	5
6.1 Jak nazywać commity	5
7. Przydatne komendy.....	6
8. .gitignore – czego NIE wysyłać	6
9. README – wizytówka projektu	6
10. Git w edytorze (PyCharm / VS Code).....	7
11. Koniec zajęć: usuwanie poświadczeń z komputera.....	8
11.1 Usuń poświadczenie GitHub z Windows	8
11.2 Usuń swoje dane z konfiguracji Git.....	8
11.3 Jeśli używałeś gh.....	8
11.4 Jeśli używałeś agenta (Claude Code / Codex / Gemini)	8
12. gh – GitHub z terminala (dodatek).....	9
13. Zasady w tej klasie	9
14. Zadania	10
Zadanie 1: Pierwsze repo	10
Zadanie 2: Zmiany	10
Zadanie 3: GitHub.....	10
Zadanie 4: README i .gitignore	10
Zadanie 5: Klonowanie	10
Zadanie 6: Z edytora.....	10
Zadanie 7: Sprzątanie.....	10

1. Po co to w ogóle jest?

Wyobraź sobie, że piszesz projekt i masz na pulpicie:

```
projekt.py
projekt_v2.py
projekt_v2_poprawiony.py
projekt_FINAL.py
projekt_FINAL_naprawde.py
```

Git rozwiązuje ten problem. Zapisuje **historię zmian** w projekcie – każdy „zapis” to **commit**. Dzięki temu możesz:

- cofnąć się do dowolnej wcześniejszej wersji,
- zobaczyć **co** się zmieniło i **kiedy**,
- pracować nad jednym projektem z innymi ludźmi bez nadpisywania sobie plików.

Git to program na Twoim komputerze. **GitHub** to strona w internecie, na której trzymasz kopię projektu (i którą pokazujesz na rozmowie o pracę).

💡 *W tej klasie każdy ma konto na GitHubie i każdy projekt z zajęć tam trafia. To jest Twoje portfolio – rekruter zobaczy je przed CV.*

2. Model w głowie: trzy miejsca

```

folder projektu (edytujesz pliki)
|   git add
v
poczekalnia (staging)
|   git commit
v
historia - repozytorium lokalne
|   git push          ^   git pull
v                       |
GitHub (kopia w chmurze)
```

1. **Folder projektu** – tu normalnie piszesz kod.
2. **Poczekalnia (staging)** – tu wskazujesz, które zmiany chcesz zapisać.
3. **Historia** – zapisane commity. Nic z tego nie znika.
4. **GitHub** – kopia historii w chmurze. **push** wysyła, **pull** pobiera.

3. Instalacja i konfiguracja

3.1 Instalacja (raz, na tym komputerze)

1. Pobierz Git: <https://git-scm.com/download/win>
2. Uruchom instalator -> klikaj Next, domyślne opcje są OK.
3. Sprawdź w terminalu (PowerShell lub Git Bash):

```
git --version
```

3.2 Konfiguracja (raz)

Git musi wiedzieć, kto robi commity. **Użyj tego samego e-maila co na GitHubie** – inaczej commity nie będą się liczyć do Twojego profilu.

```
git config --global user.name "Imię Nazwisko"
git config --global user.email "twoj@email.com"
git config --global --list          # sprawdzenie
```

⚠ **Komputery w pracowni są wspólne – po zajęciach te dane USUWASZ (rozdział 11).**

4. Konto na GitHubie

1. Wejdź na <https://github.com> -> Sign up.
2. Nazwa użytkownika: **rozsądna** – to zobaczy pracodawca. `jan-kowalski` tak, `xx_destroyer_xx` nie.
3. Potwierdź e-mail.

5. Pierwszy projekt – krok po kroku

5.1 Utwórz repozytorium na GitHubie

GitHub -> + -> New repository -> nazwa (np. **projekt-flask**) -> **nie zaznaczaj** „Add a README” -> Create repository.

5.2 W folderze projektu na komputerze

```
cd sciezka/do/projektu
git init                    # włącz Git w tym folderze
git add .                  # wrzuć wszystko do poczekalni
git commit -m "Pierwsza wersja projektu" # zapisz w historii
git branch -M main         # nazwa głównej gałęzi
git remote add origin https://github.com/TWOJ_LOGIN/projekt-flask.git
git push -u origin main    # wyślij na GitHub
```

Przy pierwszym **push** otworzy się przeglądarka – zaloguj się i zezwól na dostęp. Później już nie pyta.

5.3 Sprawdź

Odśwież stronę repozytorium na GitHubie. Pliki są? Działa.

6. Codzienna pętla

To robisz na **każdych** zajęciach:

```
git status                 # co się zmieniło?
git add .                  # do poczekalni
git commit -m "Co zrobiłem" # zapis
git push                   # na GitHub
```

Tyle. Cztery komendy. Reszta to dodatki.

6.1 Jak nazywać commity

Commit ma mówić **co** zrobiłeś. Ktoś czytający historię (np. nauczyciel) ma zrozumieć bez otwierania kodu.

Dobrze	Źle
Dodano formularz logowania	zmiany
Naprawiono błąd przy pustym koszyku	asdf
Model Produkt + migracja	działa
README: instrukcja uruchomienia	final final

💡 *Jeden commit = jedna rzecz. Kilka małych commitów jest lepsze niż jeden wielki „dodałem wszystko”.*

7. Przydatne komendy

Komenda	Co robi
<code>git status</code>	co jest zmienione / w poczekalni
<code>git log --oneline</code>	historia w jednej linii na commit
<code>git diff</code>	dokładnie które linie się zmieniły
<code>git pull</code>	pobierz zmiany z GitHuba
<code>git clone URL</code>	pobierz cudze repozytorium
<code>git restore plik.py</code>	cofnij niezapisane zmiany w pliku
<code>git show HASH</code>	pokaż, co zmienił konkretny commit

8. .gitignore – czego NIE wysyłać

Niektóre pliki nie powinny trafiać do repozytorium: cache, środowisko wirtualne, hasła, ustawienia edytora. Plik `.gitignore` w głównym folderze mówi Gitowi, co pomijać.

Projekt w Pythonie:

```
__pycache__/  
*.pyc  
venv/  
.env  
.idea/  
.vscode/  
db.sqlite3
```

Projekt w React / Node:

```
node_modules/  
dist/  
.env
```

⚠ Nigdy nie commituj pliku z hasłami, kluczami API ani node_modules. Jeśli już wysłałeś hasło na GitHub – zmień je. Usunięcie z repozytorium nie wystarczy, zostaje w historii.

9. README – wizytówka projektu

Każde repozytorium ma mieć plik `README.md`. Minimum:

```
# Nazwa projektu  
  
Jedno zdanie: co to robi.  
  
## Technologie  
Python 3.12, Flask, SQLite  
  
## Uruchomienie
```

```
pip install -r requirements.txt  
python app.py
```

```
## Autor  
Imię Nazwisko, 4TP
```

To jest **dokumentacja** – i tak ją oceniam.

10. Git w edytorze (PyCharm / VS Code)

Wszystko powyżej da się wyklikać. VS Code ma zakładkę Source Control (ikona z gałkami), PyCharm ma menu Git:

- **Ctrl + K** – okno Commit (add + commit)
- **Ctrl + Shift + K** – Push
- **Ctrl + T** – Pull
- **Alt + 9** – panel Git (historia)

Ale komendy w terminalu **musisz** znać – na rozmowie o pracę nikt nie zapyta, gdzie jest przycisk.

11. Koniec zajęć: usuwanie poświadczeń z komputera

⚠ **Komputery w pracowni są wspólne. Po pierwszym push Windows zapamiętuje Twoje logowanie do GitHuba. Jeśli go nie usuniesz, następna osoba przy tym komputerze wypchnie swoje (albo cudze) pliki na Twoje konto, pod Twoim nazwiskiem.**

Dlatego na koniec **każdy**ch zajęć:

11.1 Usuń poświadczenie GitHub z Windows

Naciśnij **Win + R**, wpisz:

```
control /name Microsoft.CredentialManager
```

-> **Poświadczenia systemu Windows** -> znajdź wpis `git:https://github.com` -> **Usuń**.

To samo z terminala (PowerShell), jeśli wolisz:

```
cmdkey /delete:git:https://github.com
```

Sprawdzenie, czy nic nie zostało:

```
cmdkey /list | findstr github
```

Pusta odpowiedź = OK.

11.2 Usuń swoje dane z konfiguracji Git

```
git config --global --unset user.name  
git config --global --unset user.email
```

Inaczej commity następnej osoby będą podpisane Twoim nazwiskiem.

11.3 Jeśli używałeś gh

```
gh auth logout
```

11.4 Jeśli używałeś agenta (Claude Code / Codex / Gemini)

W sesji agenta wpisz `/logout` (Claude Code, Codex) albo usuń folder z tokenem: `C:\Users\NAZWA\.claude, .codex, .gemini`.

💡 *Na początku każdych zajęć: `git config user.name` i `user.email` od nowa. Przy pierwszym push znowu zaloguje Cię przez przeglądarkę. To 30 sekund – a chroni Twoje konto.*

Przy okazji sprawdź: jeśli ktoś przed Tobą nie wylogował się z GitHuba w przeglądarce – nie korzystaj z tego, tylko go wyloguj. Zasada działa w obie strony.

12. gh – GitHub z terminala (dodatek)

GitHub ma własne narzędzie terminalowe, **gh**. Nie jest konieczne, ale przyspiesza kilka rzeczy.

```
winget install GitHub.cli      # instalacja (PowerShell)
gh auth login                  # logowanie (raz)
gh repo create projekt-flask --public --source=. --push              # tworzy repo na GitHubie i pushuje
gh repo view --web              # otwórz repo w przeglądarce
gh issue create                  # załóż zgłoszenie (błąd / zadanie)
```

Z **gh** krok „utwórz repo na stronie, skopiuj link, remote add” zamienia się w jedną komendę.

13. Zasady w tej klasie

1. Każdy projekt z zajęć -> osobne repozytorium na Twoim GitHubie.
2. Na koniec każdego zajęcia -> commit + push. Brak pusha = brak dowodu, że coś zrobiłeś.
3. Każde repo ma **README.md** i **.gitignore**.
4. Możesz używać LLM (Claude, ChatGPT, Gemini) do pisania kodu. **Ale na kolejnych zajęciach dostajesz pytania o ten kod.** Jeśli wiesz, co robi każda linijka – super, nie ma znaczenia, kto ją napisał. Jeśli nie wiesz – do poprawy.
5. Historia commitów jest widoczna. Jeden commit na 500 linii w 3 minuty przed dzwonkiem mówi mi więcej niż Ty.
6. Na koniec zajęć **usuwasz poświadczenia GitHub z komputera** (rozdział 11). Jeśli ktoś wypchnie coś na Twoje konto, bo zostawiłeś logowanie – to Twój problem, nie jego.

14. Zadania

Zadanie 1: Pierwsze repo

Utwórz folder `git-start`, zainicjuj repo, dodaj `hello.py` z `print("Hello Git!")`, zrób commit. Sprawdź `git log`.

Zadanie 2: Zmiany

Dopisz drugą linię do `hello.py`, sprawdź `git status` i `git diff`, zrób commit. Dodaj nowy plik `funkcje.py` z dowolną funkcją, kolejny commit. W `git log --oneline` mają być 3 commity.

Zadanie 3: GitHub

Załącz konto, utwórz repozytorium `git-start`, połącz i wypchnij. Sprawdź w przeglądarce.

Zadanie 4: README i .gitignore

Dodaj do repo `README.md` (według wzoru z rozdziału 9) i `.gitignore`. Commit, push.

Zadanie 5: Klonowanie

Sklonuj dowolne publiczne repozytorium (np. jakiś projekt w Pythonie). Obejrzyj `git log --oneline`. Znajdź commit, który naprawia jakiś błąd, i zobacz co zmienił: `git show <hash>`.

Zadanie 6: Z edytora

Zrób commit i push z poziomu VS Code lub PyCharma, bez terminala. Potem sprawdź w terminalu `git log`, czy commit tam jest.

Zadanie 7: Sprzątanie

Wykonaj procedurę z rozdziału 11. Pokaż nauczycielowi pusty wynik `cmdkey /list | findstr github`.